AWS Lab: CloudWatch

VPC and EC2 Service

3rd November 2026

1. Aim

To monitor the CPU utilization of an Amazon EC2 instance using Amazon CloudWatch, configure a threshold-based alarm, and associate the alarm with an Amazon Simple Notification Service (SNS) topic so that an email notification can be sent when the alarm enters the *In alarm* state. An EC2 stop action is also selected in the alarm configuration.

2. Objectives

1. Understand the role of Amazon CloudWatch in monitoring AWS resources.
2. Locate and graph the EC2 `CPUUtilization` metric.
3. Create an SNS Standard topic and confirm an email subscription.
4. Configure a CloudWatch alarm with a static CPU utilization threshold of 70%.
5. Configure an SNS email notification for the alarm's *In alarm* state.
6. Examine the EC2 stop action available through CloudWatch alarms.

3. Requirements

- AWS account with access to Amazon EC2, Amazon CloudWatch, and Amazon SNS.
- An EC2 instance publishing monitoring metrics to CloudWatch.
- A valid email address for receiving SNS notifications.
- Web browser and internet connectivity.

4. Theory

4.1 Amazon CloudWatch

Amazon CloudWatch is an AWS monitoring and observability service. It collects metrics and logs from AWS resources and applications. Users can view these measurements in graphs, create alarms based on metric thresholds, and configure actions when an alarm changes state.

4.2 CPU Utilization Metric

The EC2 metric `CPUUtilization` represents the percentage of allocated compute capacity being used by the instance. A high value may indicate that the instance is under heavy processor load. CloudWatch plots this metric over time and can evaluate it against a user-defined threshold.

4.3 CloudWatch Alarm

A CloudWatch alarm evaluates a metric against a configured condition. Its main states are:

- **OK:** The metric is within the configured condition.
- **ALARM:** The metric has breached the configured condition for the required evaluation period.
- **INSUFFICIENT_DATA:** There is not enough metric data to determine the state.

4.4 Amazon Simple Notification Service (SNS)

Amazon SNS is a managed publish-and-subscribe messaging service. A publisher sends a message to an SNS topic, and SNS delivers it to the topic's subscribers. In this experiment, an email endpoint is subscribed to a Standard topic named `MyCwAlarm`. The email recipient must confirm the subscription before receiving topic notifications.

5. Configuration Used

Parameter	Value shown in screenshots
AWS Region	US West (Oregon)
SNS topic name	`MyCwAlarm`
SNS topic type	Standard
SNS subscription protocol	Email
CloudWatch namespace	`AWS/EC2`
EC2 instance name	Stress Test
EC2 instance ID	`i-0aa0e99a923d9d8ea`
Metric	`CPUUtilization`
Statistic	Average
Threshold condition	Greater than 70%
Period in final alarm preview	1 minute
Alarm name	`LabCPUUtilizationAlarm`
SNS notification	Topic `MyCwAlarm`, In alarm state
EC2 action	Stop this instance, In alarm state

Configuration note: The metric-condition screen shows a 5-minute period, whereas the final preview screen shows a 1-minute period. The final preview value is recorded in the configuration table. The supplied screenshots document the workflow through the preview stage; they do not include a final confirmation that the CloudWatch alarm itself was created.

6. Procedure

6.1 Part A: Create an SNS Topic

Step 1: Open Amazon SNS and navigate to Topics.

Sign in to the AWS Management Console. Search for **Amazon SNS** and open the service. From the navigation pane, select **Topics**. The Topics page displays the SNS topics available in the selected AWS Region. Select **Create topic** to begin creating a topic.

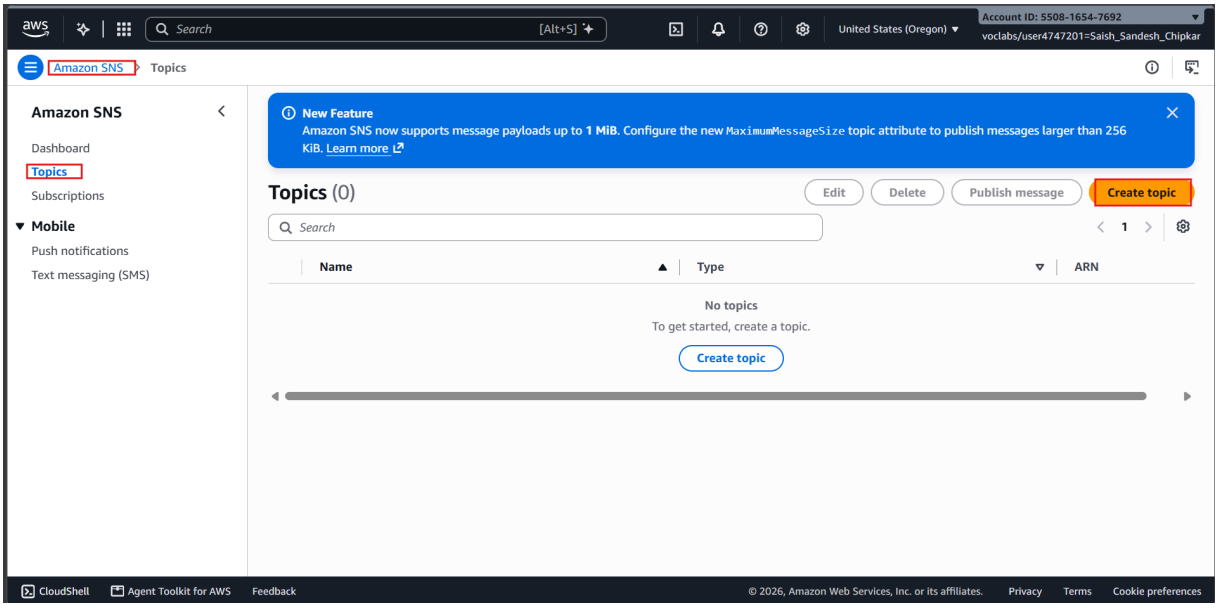


Figure 1: Amazon SNS Topics page before creating a topic.

Step 2: Select the Topic Type and Enter the Topic Name.

On the Create topic page, AWS SNS provides two types of topics:

- **Standard Topic:** Provides best-effort message ordering and at-least-once message delivery. It supports multiple subscription protocols, including Email, Amazon SQS, AWS Lambda, HTTP/S, SMS and mobile application endpoints. It is suitable for applications where high message throughput and flexible message delivery are important.
- **FIFO (First-In, First-Out) Topic:** Preserves the exact order of messages and supports exactly-once message delivery within its deduplication scope. It is suitable for applications where message ordering and duplicate prevention are critical. FIFO topics support Amazon SQS subscriptions.

For this experiment, select **Standard** because the objective is to receive CloudWatch alarm notifications through an email subscription. Standard topics support email notifications and are suitable for this monitoring use case.

Enter `MyCwAlarm` in the Name field. Keep the remaining settings at their default values and proceed with creating the topic.

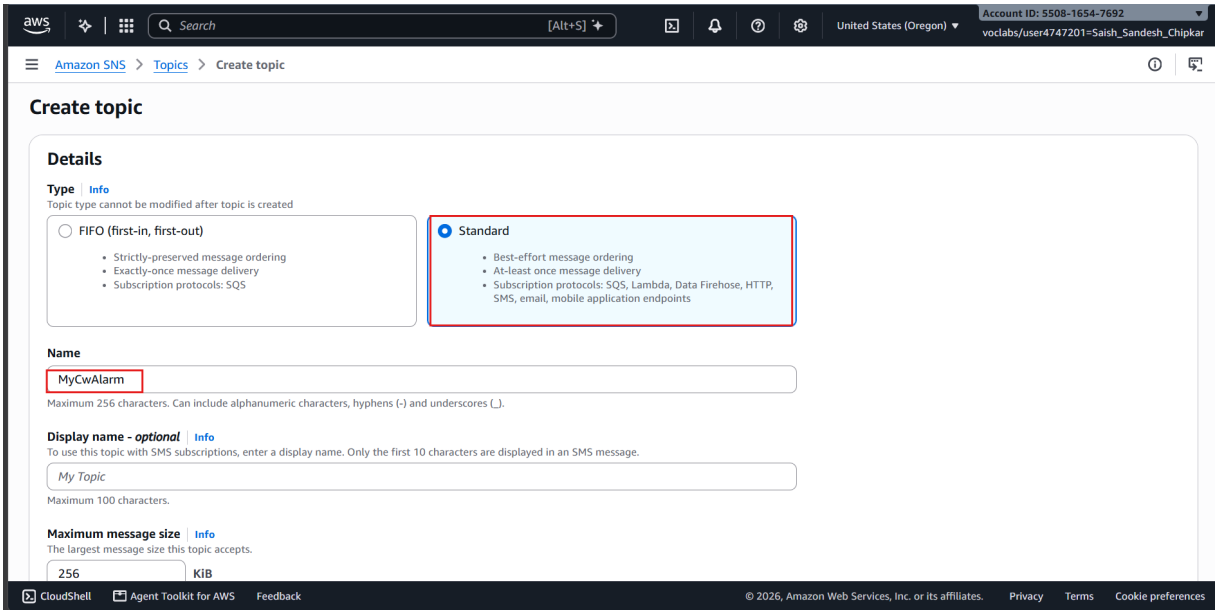


Figure 2: Selecting Standard or FIFO topic type and entering the SNS topic name.

Step 3: Verify that the SNS topic was created.

After creation, the SNS console displays a success message and opens the topic details page. Verify that the topic name is `MyCwAlarm`. This topic will later be selected as the notification destination for the CloudWatch alarm. From the Subscriptions section, select **Create subscription**.

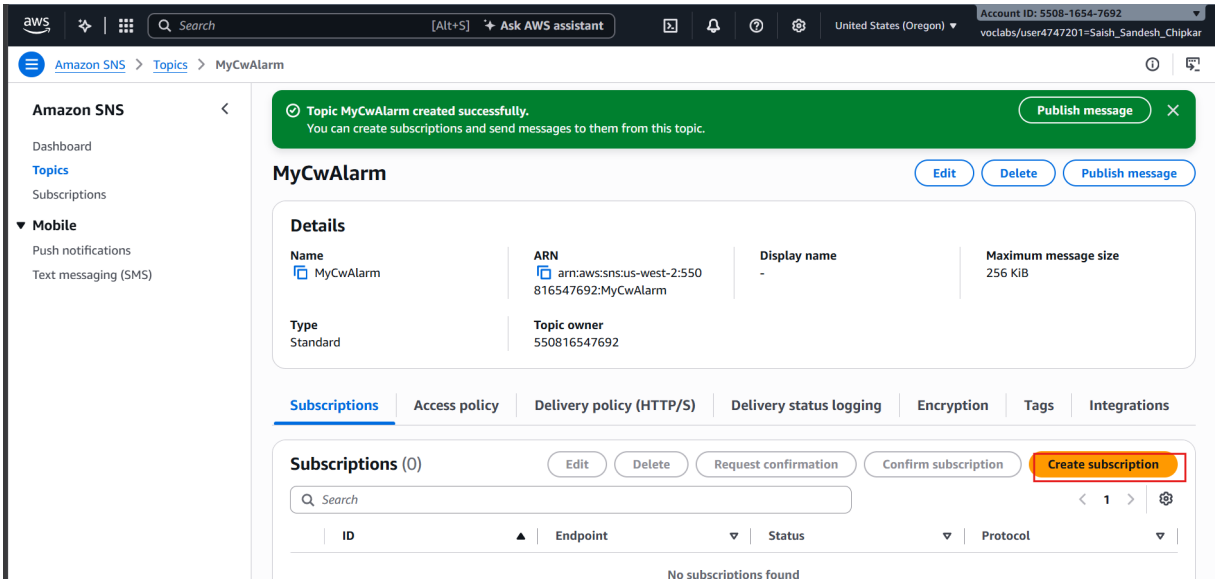


Figure 3: SNS topic MyCwAlarm created successfully; the subscription can now be created.

6.2 Part B: Create and Confirm the SNS Email Subscription

Step 4: Enter the subscription details.

On the Create subscription page, select the ARN of the `MyCwAlarm` topic. Set the protocol to **Email**, then enter the email address that should receive alarm notifications. Select **Create**

subscription. The endpoint must be an email address that the recipient can access.

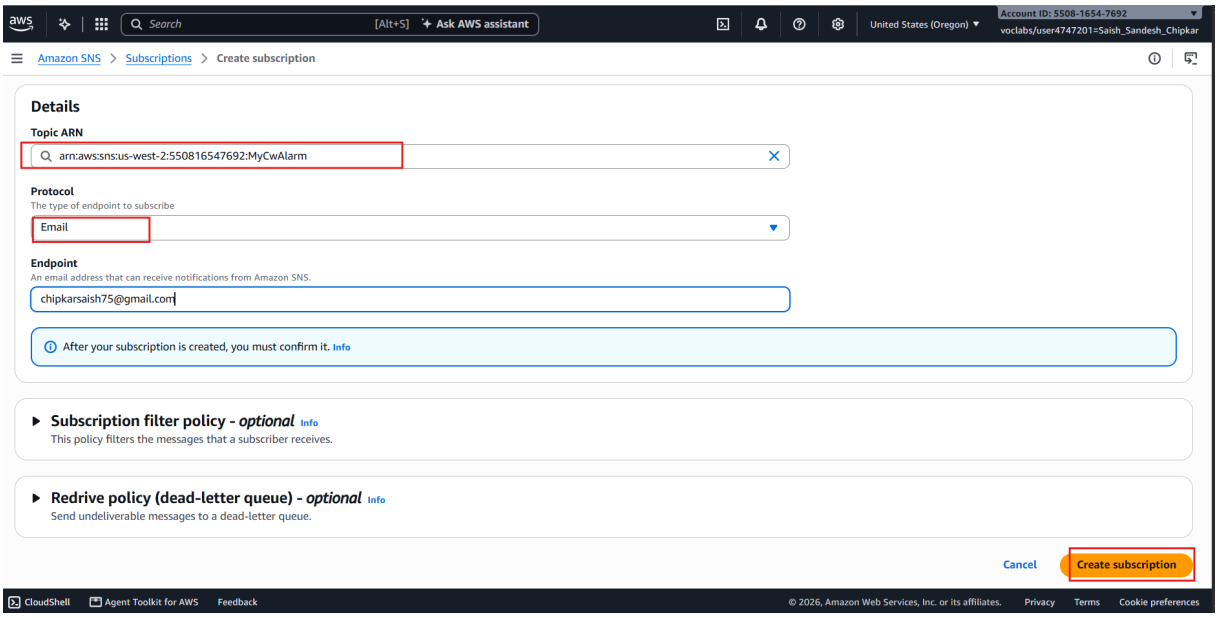


Figure 4: Entering the SNS topic ARN, Email protocol, and email endpoint.

Step 5: Check the initial subscription status.

After the subscription is created, SNS displays its subscription details. The status is initially shown as *Pending confirmation*. This is expected: SNS requires the email recipient to confirm the subscription before messages can be delivered to that endpoint.

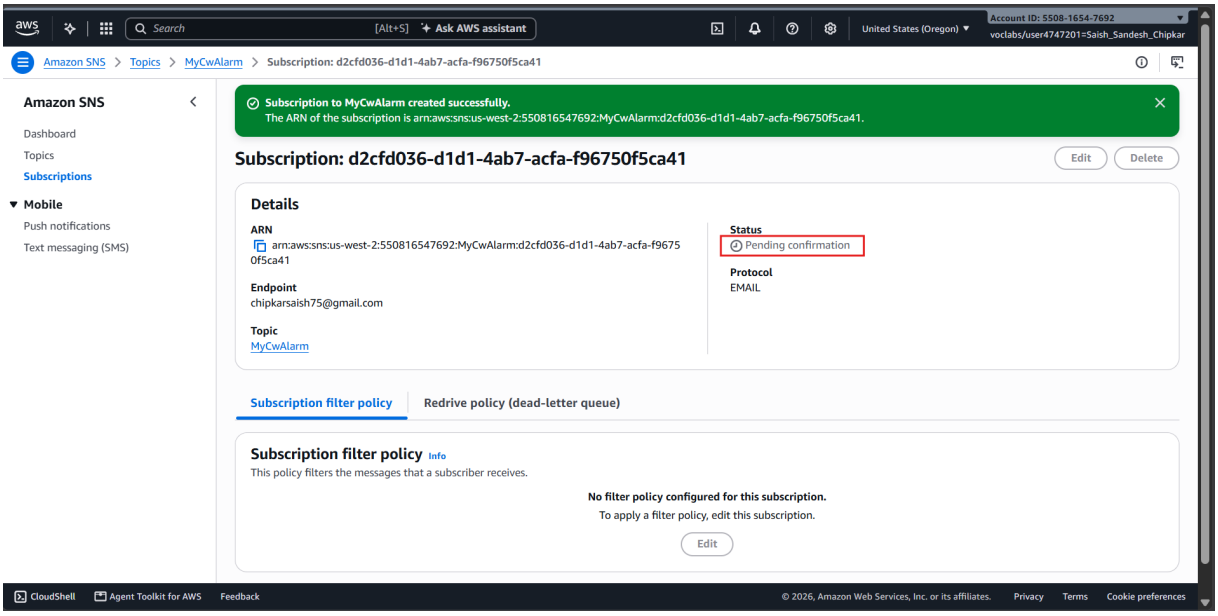


Figure 5: New SNS email subscription with Pending confirmation status.

Step 6: Open the AWS subscription confirmation email.

Open the recipient's email inbox and locate the message sent by AWS Notifications with the subject *AWS Notification - Subscription Confirmation*. Check that the message refers to the `MyCwAlarm` topic, then select the **Confirm subscription** link.

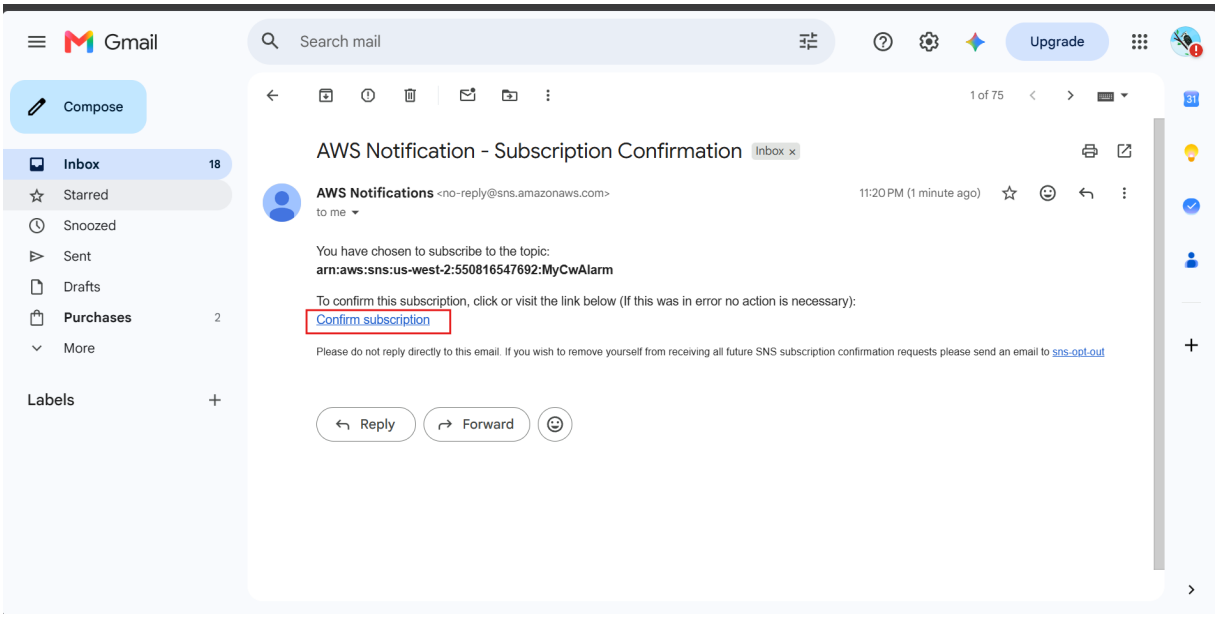


Figure 6: AWS SNS subscription confirmation email containing the confirmation link.

Step 7: Confirm the subscription on the AWS page.

The confirmation link opens the Amazon SNS confirmation page. Select the **Confirm Subscription** button to approve the email endpoint's subscription to the topic.



Figure 7: Amazon SNS page asking the recipient to confirm the subscription.

Step 8: Verify the confirmation result.

SNS displays a success message stating that the subscription has been confirmed. This indicates that the email endpoint has accepted the subscription request and can receive messages published to the topic.

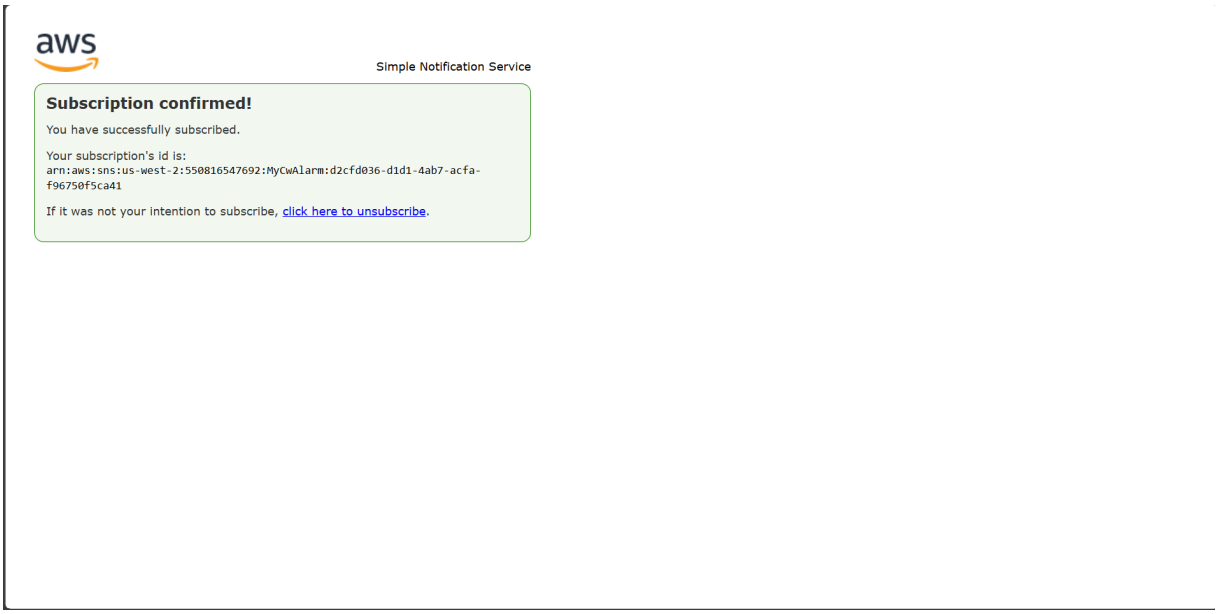


Figure 8: SNS confirmation page showing that the subscription was successful.

Step 9: Verify the status in the SNS console.

Return to the SNS console and open the subscription details. Confirm that the status is now *Confirmed* and that the protocol is `EMAIL`. This verification is important because an unconfirmed email subscription will not receive the topic's notifications.

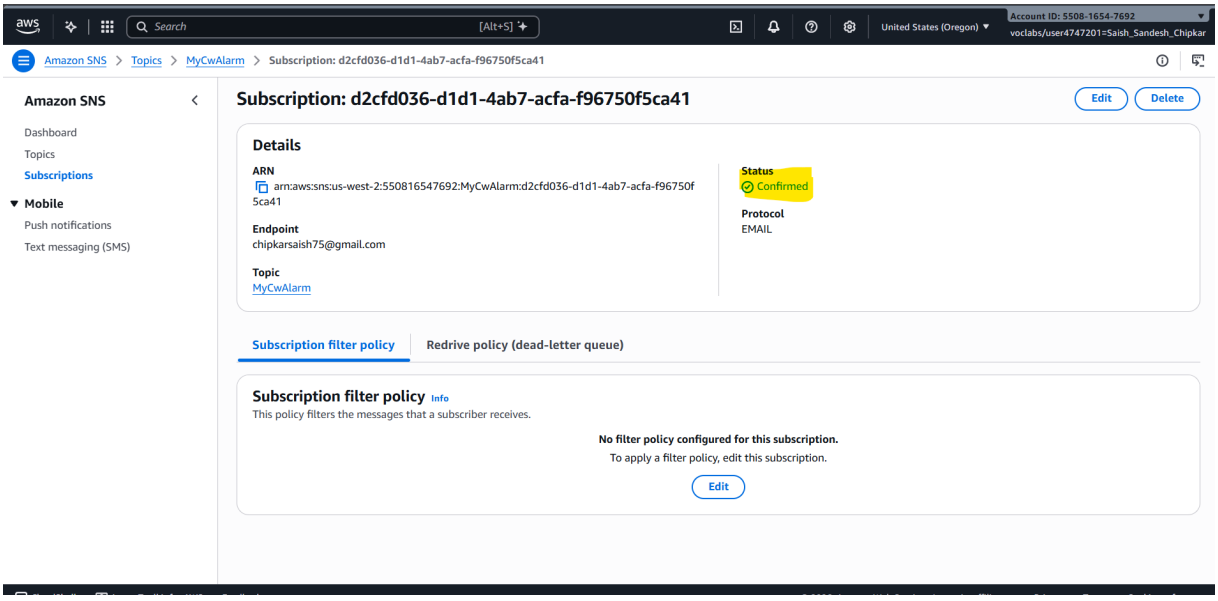


Figure 9: SNS console showing the email subscription status as Confirmed.

6.3 Part C: Locate the EC2 CPU Utilization Metric

Step 10: Open the CloudWatch service.

Search for **CloudWatch** in the AWS Management Console and open it. From the CloudWatch navigation pane, expand **Metrics** and select **Classic metrics**. This opens the metric browsing interface used to find the EC2 performance metrics.

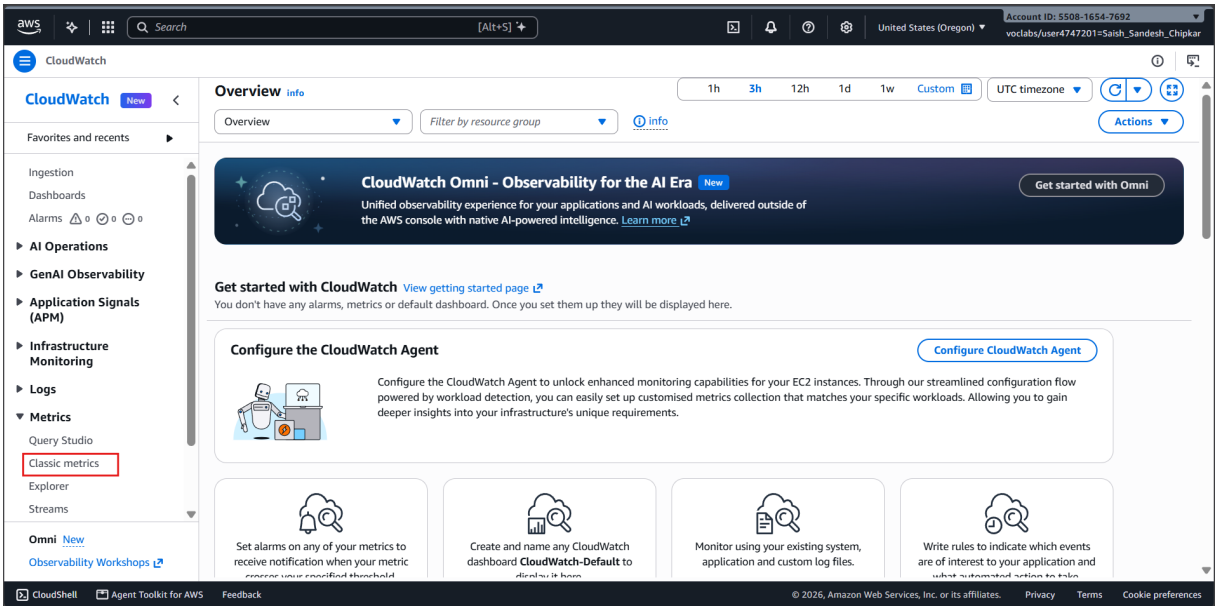


Figure 10: CloudWatch overview with Classic metrics selected in the navigation pane.

Step 11: Browse the available metric namespaces.

On the Metrics page, locate the **EC2** namespace. AWS services publish metrics under namespaces; the EC2 namespace contains metrics associated with EC2 instances, such as CPU utilization and EBS-related measurements. Select **EC2** to browse its metrics.

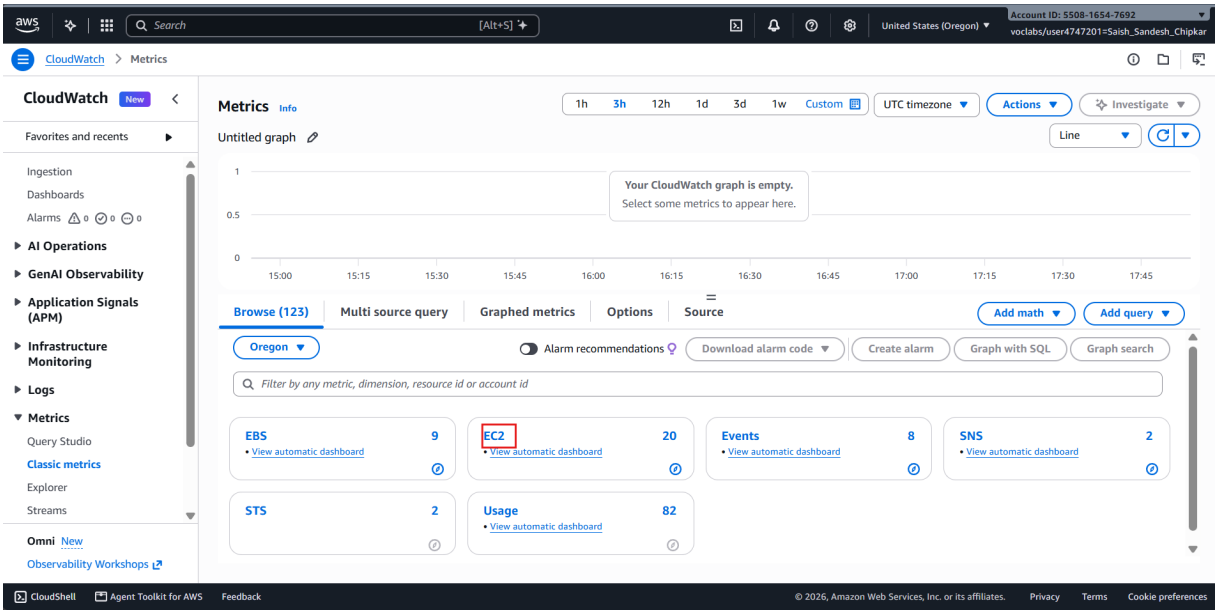


Figure 11: CloudWatch Metrics page showing the EC2 metric namespace.

Step 12: Select CPUUtilization for the required instance.

Find the instance named `Stress Test` and select its `CPUUtilization` metric. The metric is associated with the instance ID shown in the console. Once selected, CloudWatch displays the CPU utilization graph for the chosen time range. The graph provides a time-based view of processor usage.

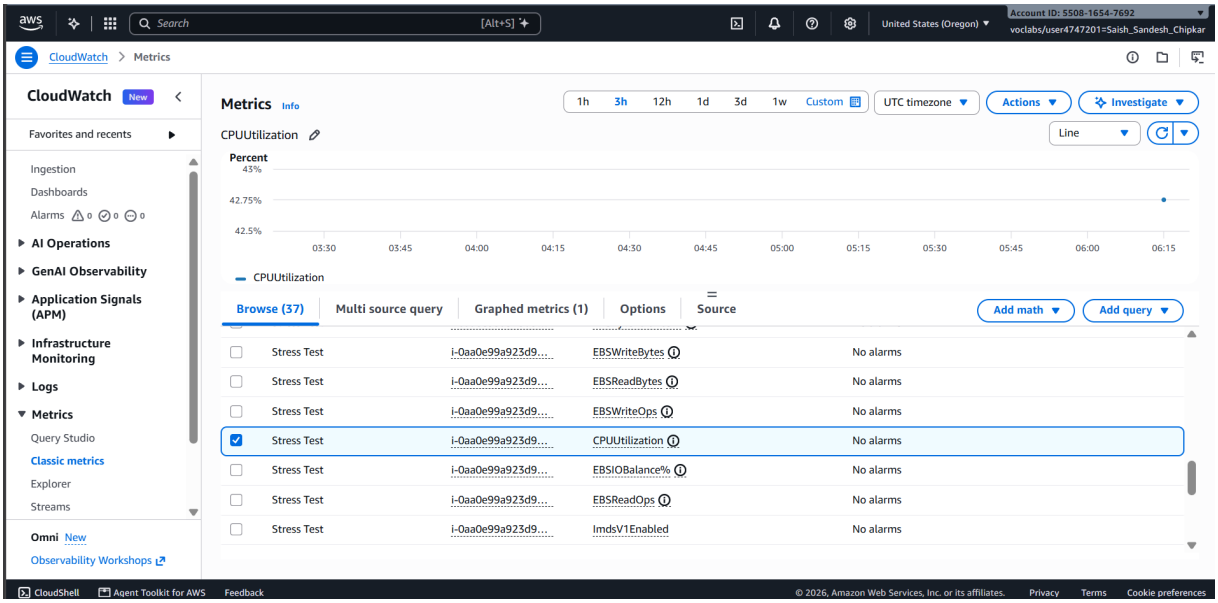


Figure 12: CPUUtilization metric selected for the Stress Test EC2 instance.

6.4 Part D: Configure the CloudWatch Alarm

Step 13: Open the Alarms section.

In the CloudWatch navigation pane, select **Alarms**. The page lists the alarms configured in the account and Region. Select **Create alarm** to define a new alarm for the EC2 CPU utilization metric.

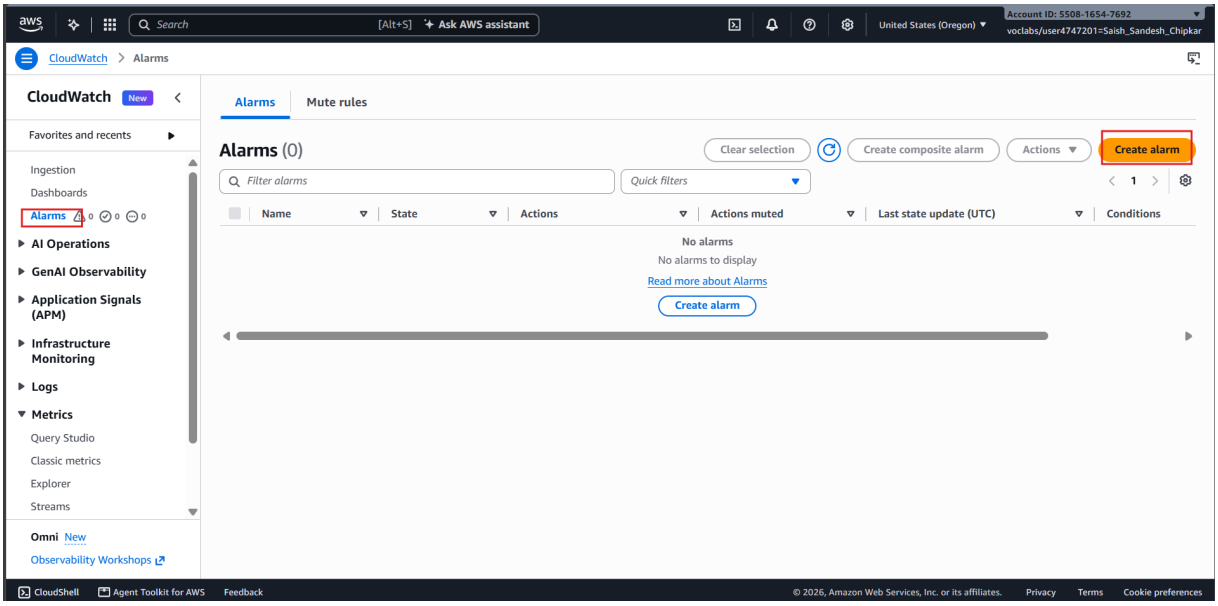


Figure 13: CloudWatch Alarms page before creating the CPU utilization alarm.

Step 14: Choose the Alarm Data Source and Metric Type.

In Step 1 of the CloudWatch alarm creation wizard, AWS provides three data source options:

- **Metrics:** Monitors numerical performance measurements collected by CloudWatch,

such as CPU utilization, network traffic, disk operations and memory usage when available.

- **Logs:** Allows monitoring and analyzing log data using CloudWatch Logs queries. It can be used to create alarms based on patterns or numerical results obtained from log data.
- **Alarms:** Allows the creation of composite alarms by combining the states of multiple existing CloudWatch alarms using logical conditions.

Under the Type section, two metric types are available:

- **Classic:** Provides the standard CloudWatch metric selection experience, allowing users to select predefined metrics and configure alarm conditions.
- **PromQL:** Uses Prometheus Query Language (PromQL) expressions to define metrics for alarms. It is useful when monitoring Prometheus-compatible metrics.

For this experiment, select **Metrics** as the data source and **Classic** as the metric type because the objective is to monitor the EC2 instance's CPU utilization using the standard CloudWatch metrics.

Under the Metric section, click **Select metric**. This opens the metric browser, where the required EC2 instance metric can be selected.

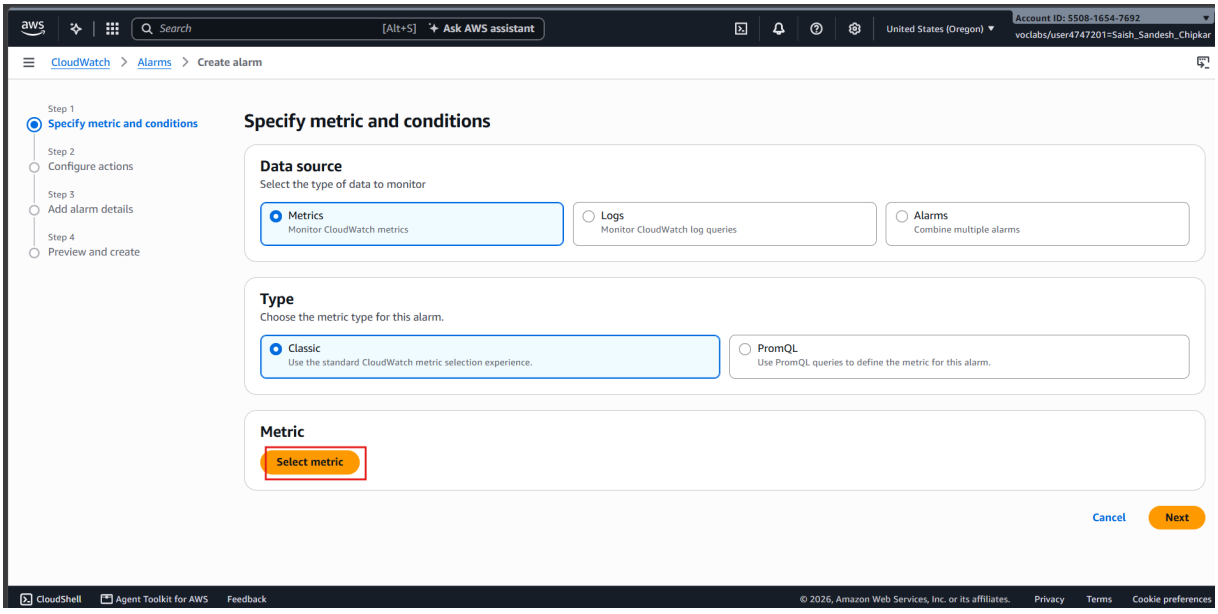


Figure 14: Selecting Metrics as the data source and Classic as the metric type.

Step 15: Choose the EC2 CPUUtilization metric.

In the metric selection window, locate the row for the `Stress Test` instance and select `CPUUtilization`. Check that the selected metric belongs to the intended instance, then select **Select metric** to return to the alarm configuration page.

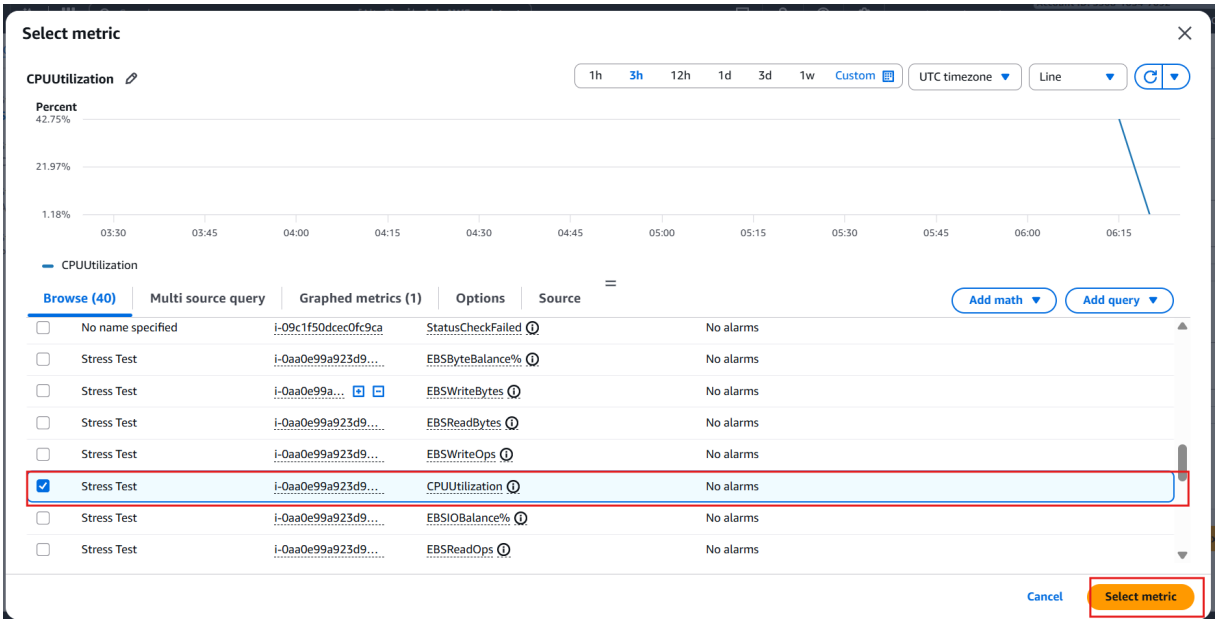


Figure 15: Selecting the CPUUtilization metric for the Stress Test instance.

Step 16: Define the CPU Utilization Threshold.

First, verify the selected metric details:

- **Namespace:** `AWS/EC2`
- **Metric Name:** `CPUUtilization`
- **Instance Name:** `Stress Test`
- **Statistic:** Average
- **Period:** 5 minutes (initial configuration).

Under the Conditions section, AWS provides two threshold types:

- **Static:** Uses a predefined numerical threshold to evaluate the selected metric.
- **Anomaly Detection:** Uses machine learning to establish an expected range of metric values and identifies values that deviate from the expected behaviour.

Select **Static** for this experiment.

Under the alarm condition, four comparison operators are available:

- **Greater ($>$):** Triggers when the metric value exceeds the specified threshold.
- **Greater/Equal ($\geq$):** Triggers when the metric value is greater than or equal to the threshold.
- **Lower/Equal ($\leq$):** Triggers when the metric value is less than or equal to the threshold.

- **Lower (<):** Triggers when the metric value falls below the specified threshold.

For this experiment, select **Greater (>)** and enter **70** as the threshold value.

Therefore, the alarm evaluates the average CPU utilization and triggers when it exceeds 70%.

The screenshot shows an initial period of 5 minutes. The period is subsequently changed to 1 minute in the final alarm configuration.

Click **Next** to continue.

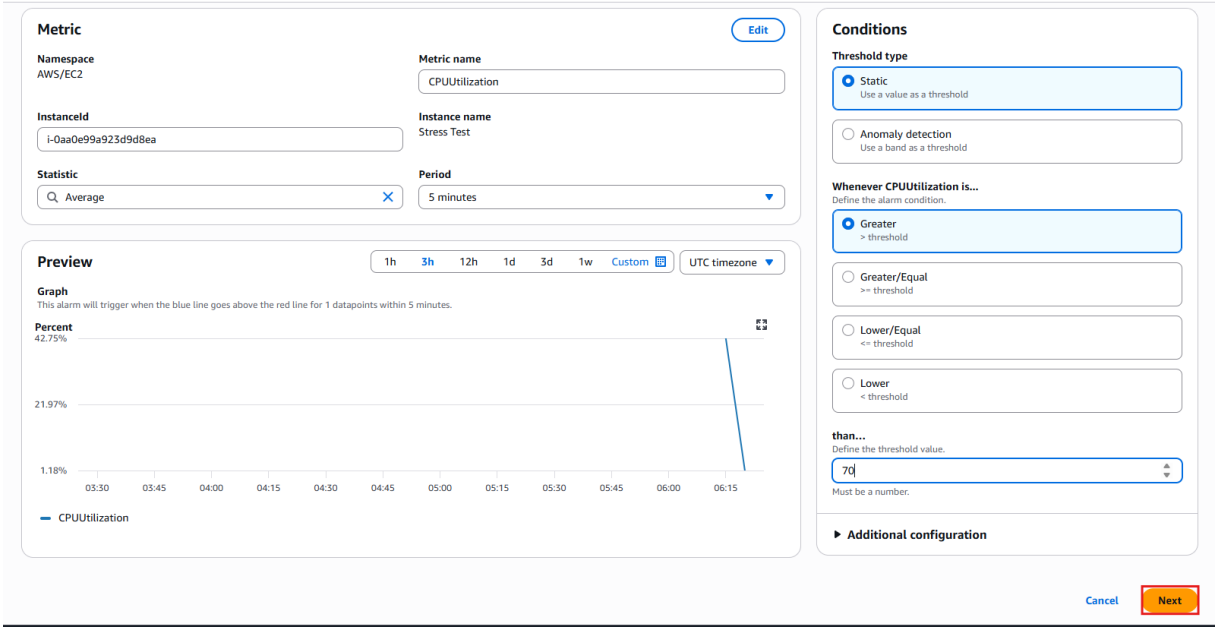


Figure 16: Configuring a static threshold of 70% for EC2 CPU utilization.

Step 17: Configure the SNS Notification Action.

In the Configure actions stage, AWS CloudWatch provides three alarm state triggers:

- **In alarm:** Triggers the action when the metric crosses the configured threshold and the alarm enters the ALARM state.
- **OK:** Triggers the action when the metric returns to the defined acceptable range and the alarm enters the OK state.
- **Insufficient data:** Triggers the action when CloudWatch does not have enough metric data to determine whether the alarm condition has been met.

For this experiment, select **In alarm** because an email notification is required when CPU utilization exceeds 70%.

Under the SNS topic selection section, three options are available:

- **Select an existing SNS topic:** Uses an SNS topic that has already been created in the AWS account.

- **Create new topic:** Creates a new SNS topic directly from the alarm configuration workflow.
- **Use topic ARN to notify other accounts:** Allows the selection of an SNS topic using its Amazon Resource Name (ARN), including a topic belonging to another AWS account, provided the necessary permissions are configured.

For this experiment, select **Select an existing SNS topic** and choose the previously created topic named `MyCwAlarm`.

Verify that the SNS topic ARN and the confirmed email endpoint are displayed correctly.

When the alarm enters the ALARM state, CloudWatch publishes a notification to the selected SNS topic, which delivers the notification to its confirmed email subscriber.

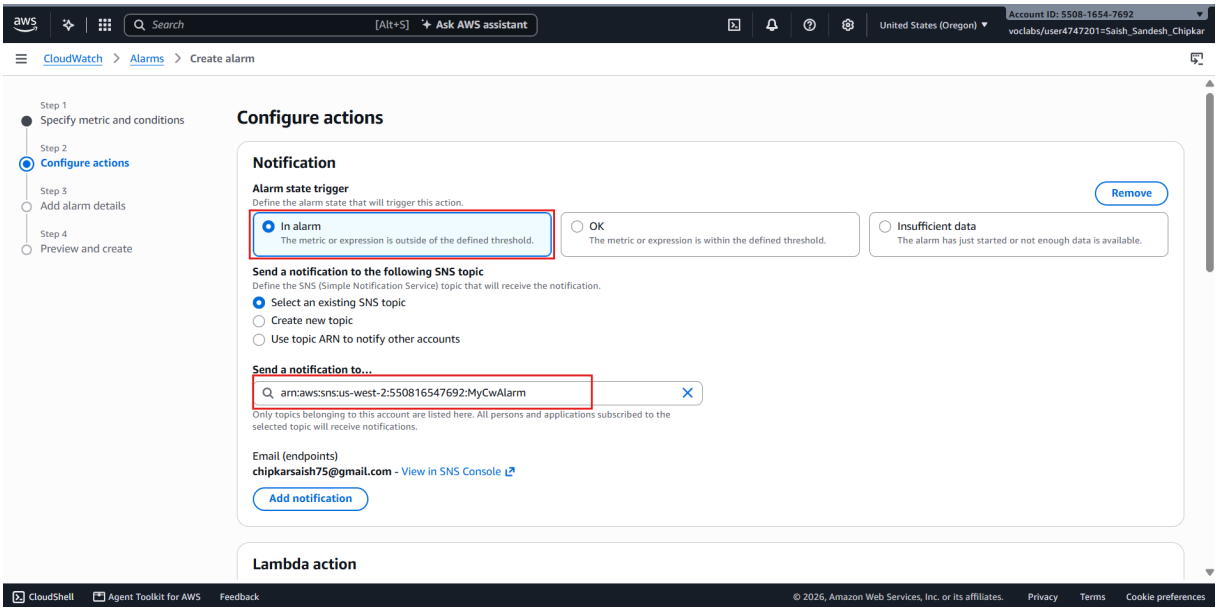


Figure 17: Configuring the In alarm trigger and selecting the existing MyCwAlarm SNS topic.

Step 18: Configure the EC2 action.

In the EC2 action section, select **In alarm** as the trigger state and choose **Stop this instance**. This configures an action for CloudWatch to request that the selected EC2 instance be stopped when the alarm enters the In alarm state. This is an operational action and can interrupt applications or tasks running on the instance, so it should only be enabled when stopping the instance is intended. Continue to the alarm details stage.

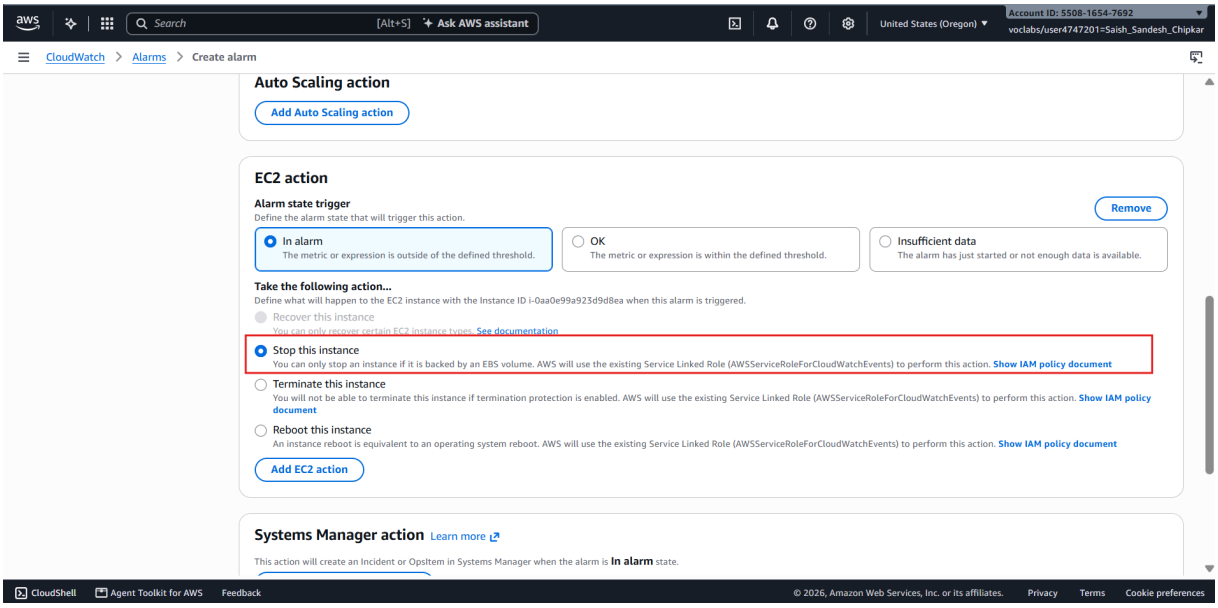


Figure 18: Selecting Stop this instance as the EC2 action for the In alarm state.

Step 19: Enter the alarm name and description.

In Add alarm details, enter `LabCPUUtilizationAlarm` as the alarm name. Add the description *CloudWatch alarm for Stress Test EC2 instance CPU utilization*. The name helps identify the alarm in the CloudWatch console, while the description explains the resource and metric being monitored. Proceed to the preview stage.

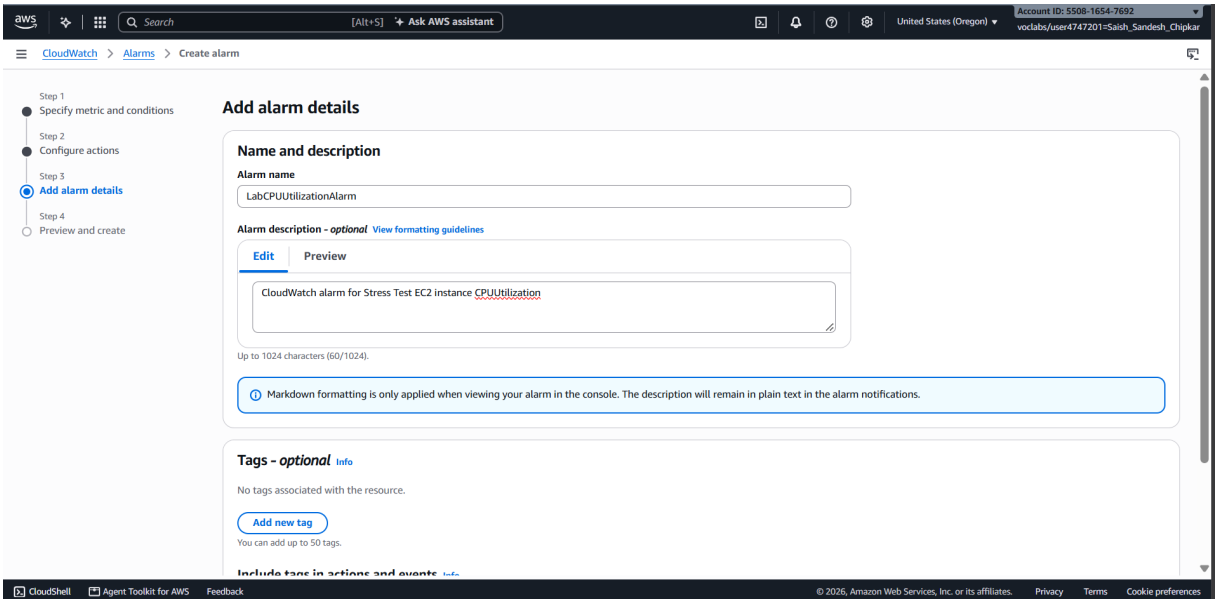


Figure 19: Entering the CloudWatch alarm name and description.

Step 20: Review the alarm configuration.

On the Preview and create page, review the metric namespace, metric name, instance ID, statistic, period, threshold, and configured actions. The supplied preview shows the `CPUUtilization` metric for the `Stress Test` instance, Average statistic, a 1-minute period, and a 70% threshold. It also reflects the selected notification and EC2 action settings from the previous stage.

The screenshot shows the preview stage; select **Create alarm** in the console to finish creating the alarm if that final action has not yet been performed.

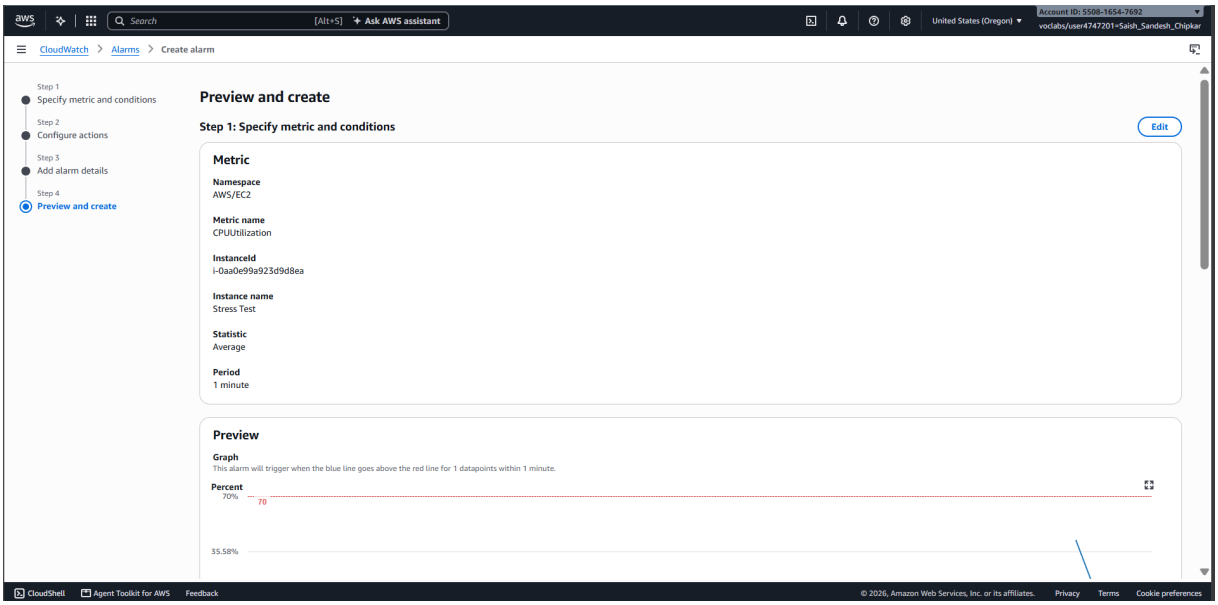


Figure 20: Final CloudWatch alarm configuration preview before creation.

7. Final Testing, Alarm Verification and Dashboard Creation

7.1 Step 21: Review Alarm Configuration

After configuring the CloudWatch alarm conditions, review the preview before creating the alarm. The selected metric is CPUUtilization, with a static threshold condition.

The alarm is configured to trigger when CPU utilization becomes greater than 70% for at least one datapoint within a one-minute period. The preview graph displays the selected threshold and the CPU utilization metric.

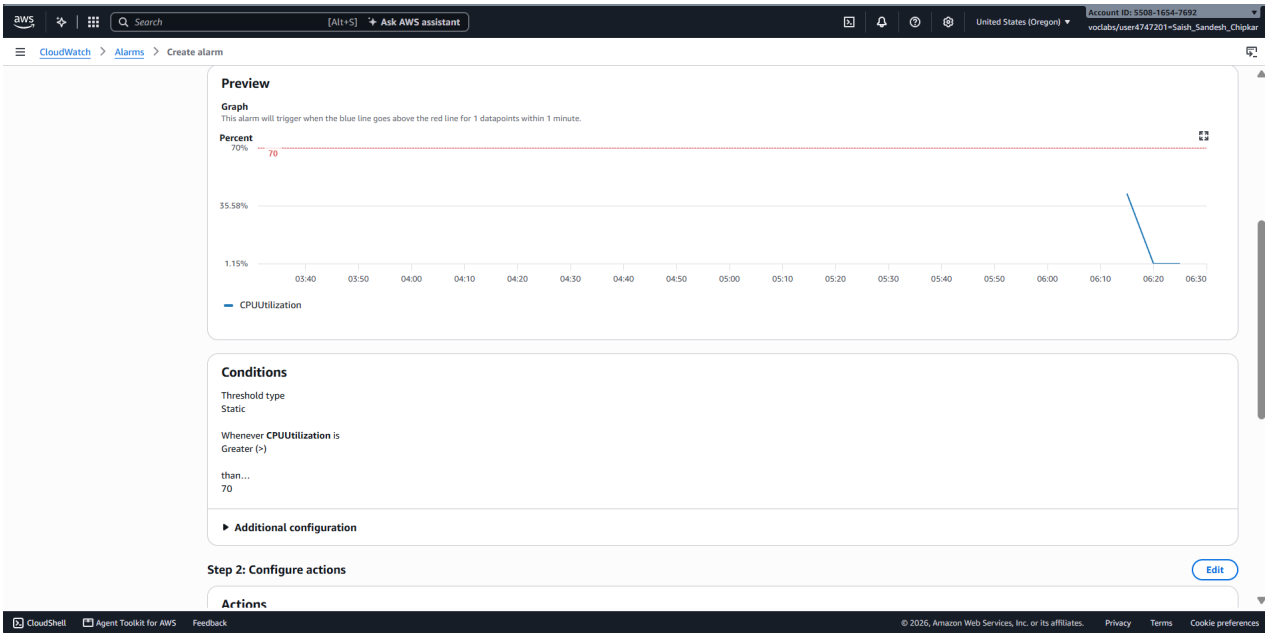


Figure 21: CloudWatch alarm configuration preview

7.2 Step 22: Review Actions and Add Alarm Details

In this step, review the notification and EC2 actions configured for the alarm.

The following actions are configured:

- **Notification Action:** When the alarm enters the ALARM state, an email notification is sent through the SNS topic named `MyCwAlarm`.
- **EC2 Action:** When the alarm enters the ALARM state, the associated EC2 instance is automatically stopped.

The alarm details are configured as follows:

- **Alarm Name:** LabCPUUtilizationAlarm
- **Description:** CloudWatch alarm for Stress Test EC2 instance CPUUtilization.

After reviewing the configuration, click on the Create alarm button.

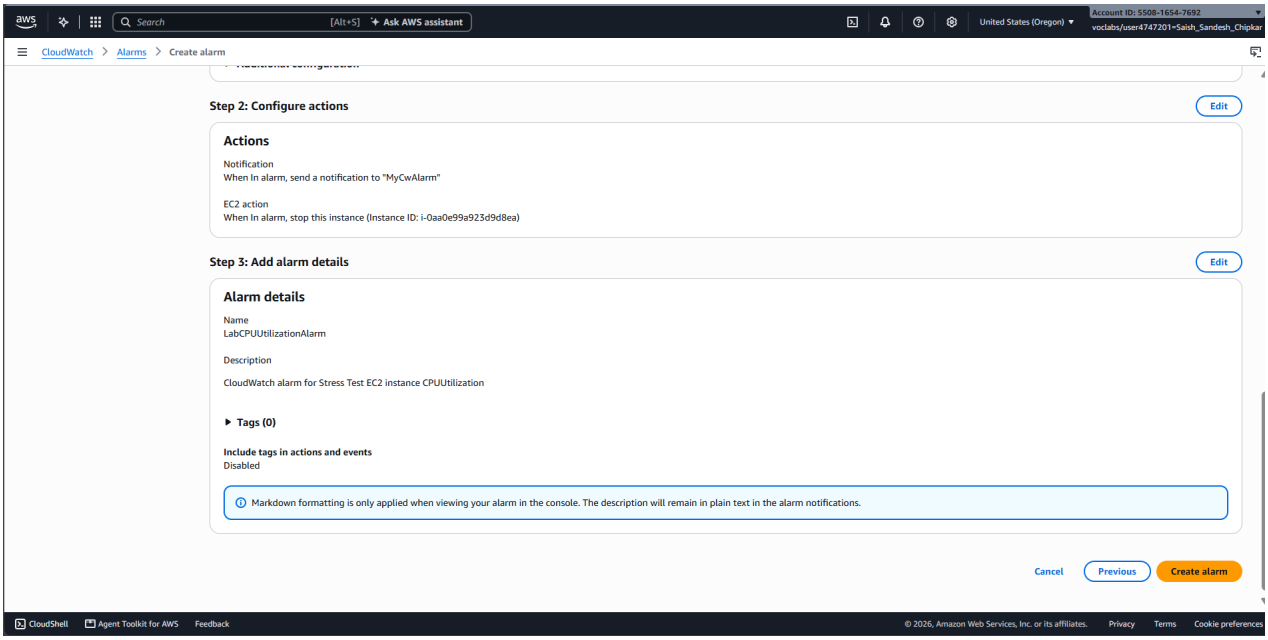


Figure 22: Reviewing alarm actions and alarm details

7.3 Step 23: Verify Successful Alarm Creation

After clicking Create alarm, AWS CloudWatch successfully creates the alarm named `LabCPUUtilizationA`

The newly created alarm appears in the CloudWatch Alarms dashboard. Initially, its state may display as Insufficient data because CloudWatch has not yet received enough metric datapoints to evaluate the configured condition.

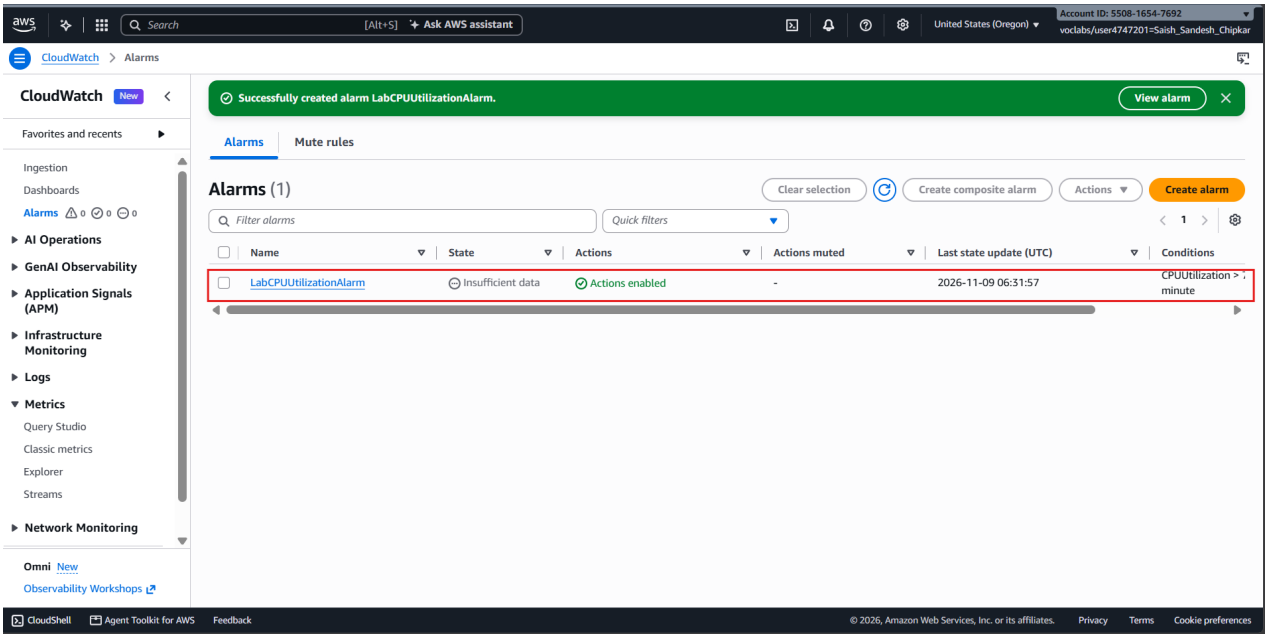


Figure 23: Successful creation of the CloudWatch alarm

7.4 Step 24: Generate CPU Load Using Stress Utility

To test the configured alarm, generate artificial CPU load on the running EC2 instance.

Connect to the EC2 instance using the terminal and execute the following command:

```
sudo stress --cpu 10 -v --timeout 400s
```

The command performs the following operations:

- `sudo`: Executes the command with administrator privileges.
- `stress`: Runs the stress-testing utility.
- `--cpu 10`: Creates 10 CPU stress workers.
- `-v`: Enables verbose output.
- `--timeout 400s`: Runs the stress test for 400 seconds.

This artificial workload increases CPU utilization so that the CloudWatch alarm condition can be tested.

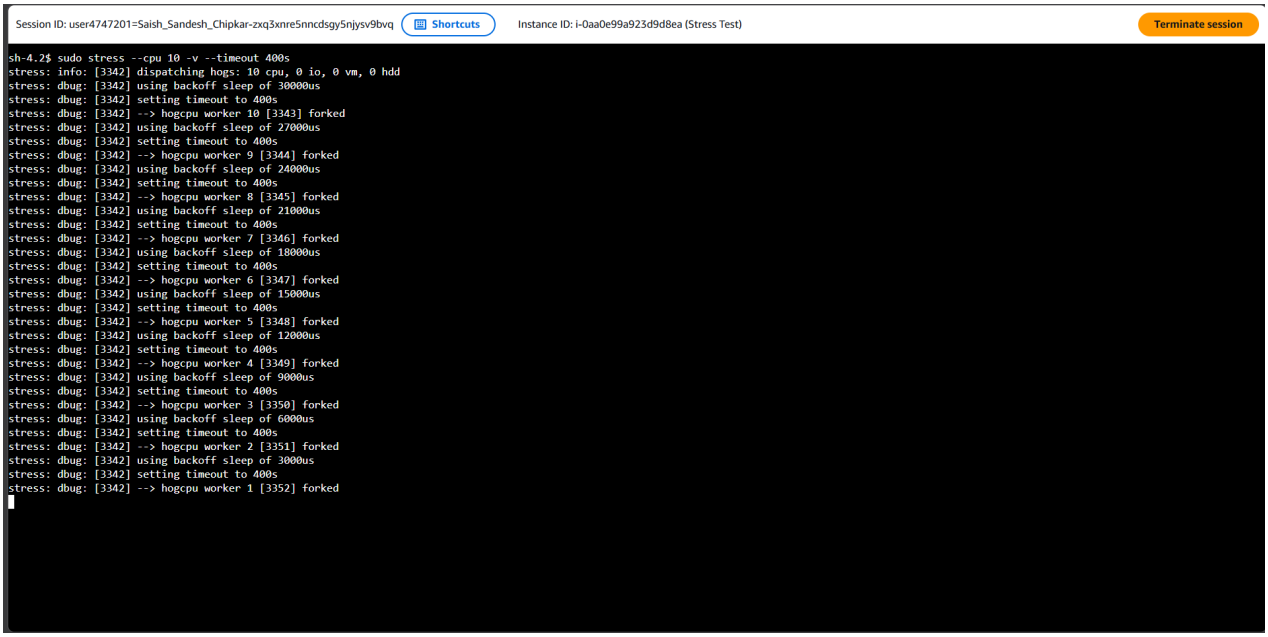


Figure 24: Executing the CPU stress test on the EC2 instance

7.5 Step 25: Monitor CPU Utilization During Stress Testing

After executing the stress command, use the Linux `top` command to monitor the CPU utilization of the EC2 instance.

The output displays the running stress processes and their CPU consumption.

The CPU utilization reaches approximately 100%, indicating that the stress workers are successfully generating a high CPU workload.

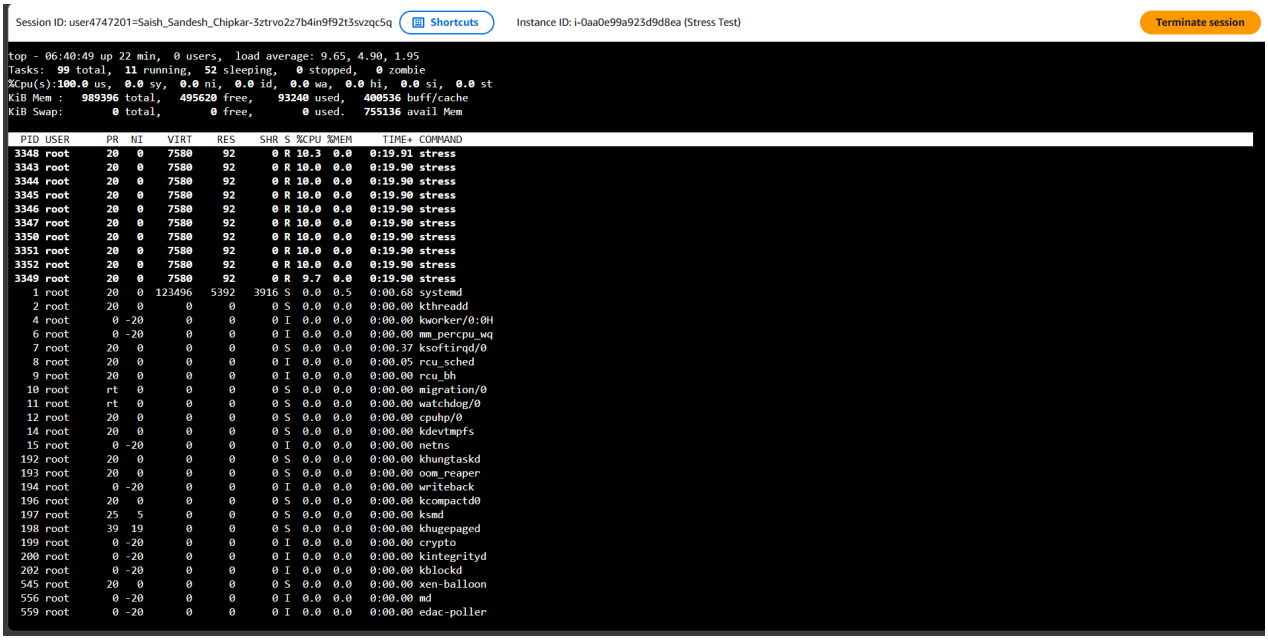


Figure 25: Monitoring CPU utilization using the top command

7.6 Step 26: Monitor the CloudWatch Alarm State

Navigate to the CloudWatch Alarms section and open `LabCPUUtilizationAlarm`.

Monitor the CPUUtilization graph to observe the metric values against the configured 70% threshold.

As the CPU utilization increases beyond the threshold, CloudWatch evaluates the incoming datapoints and changes the alarm state when the configured condition is satisfied.

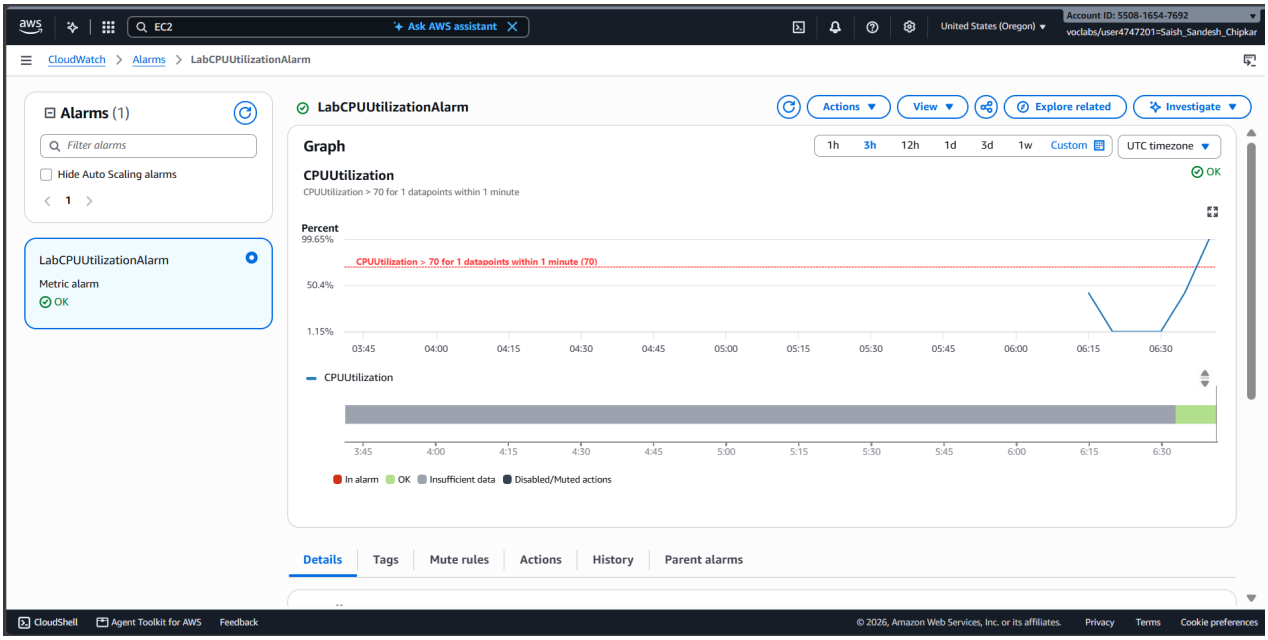


Figure 26: Monitoring CPU utilization and alarm state in CloudWatch

7.7 Step 27: Verify Email Notification Through SNS

Once the CPU utilization exceeds the configured threshold, CloudWatch changes the alarm state from OK to ALARM.

Since an SNS notification action was configured, AWS sends an email notification to the subscribed email address.

Open the registered Gmail account and verify the notification received from AWS Notifications.

The email contains the following information:

- Alarm name: `LabCPUUtilizationAlarm`
- State change: OK to ALARM
- Reason: CPU utilization exceeded the configured 70% threshold.
- AWS Region: US West (Oregon).

This confirms that the CloudWatch alarm and SNS email notification are functioning correctly.

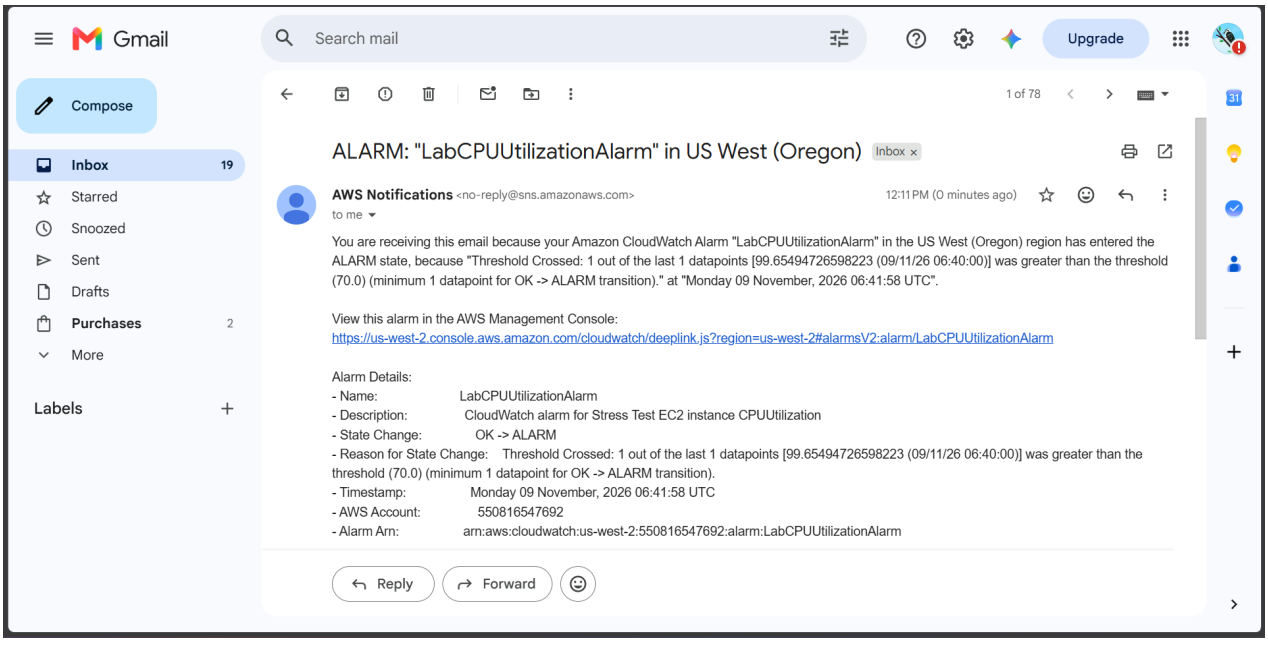


Figure 27: Email notification received from AWS CloudWatch through SNS

7.8 Step 28: Verify Automatic EC2 Instance Stop Action

After the alarm enters the ALARM state, CloudWatch executes the configured EC2 action.

Navigate to the EC2 Instances section and check the state of the associated instance named `Stress Test`.

The instance state is displayed as **Stopped**, confirming that the configured EC2 stop action was executed.

This demonstrates how CloudWatch can automatically respond to high CPU utilization by stopping an EC2 instance.

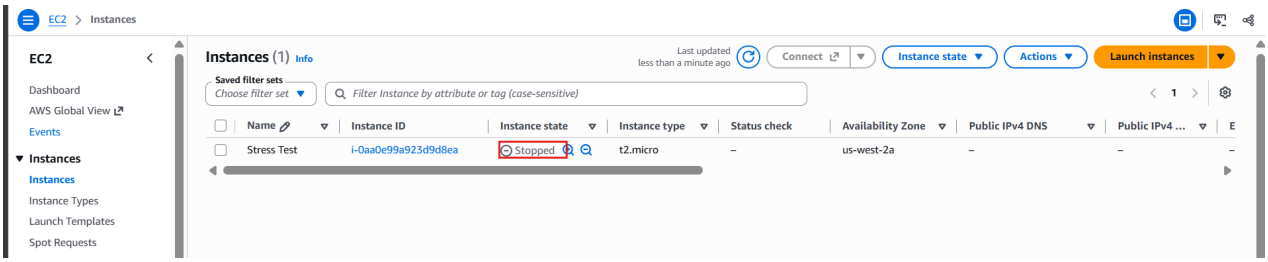


Figure 28: Verification of automatic EC2 instance stop action

7.9 Step 29: Create a CloudWatch Dashboard Widget

To visualize the CPU utilization metric in a dashboard, navigate to CloudWatch Dashboards and open the dashboard named `stresstest`.

Click on the Add widget button.

In the Add widget configuration window, select the CloudWatch data source and choose Metrics as the data type.

Select the Line widget type to visualize the CPU utilization metric over time. Click Next to proceed with metric selection.

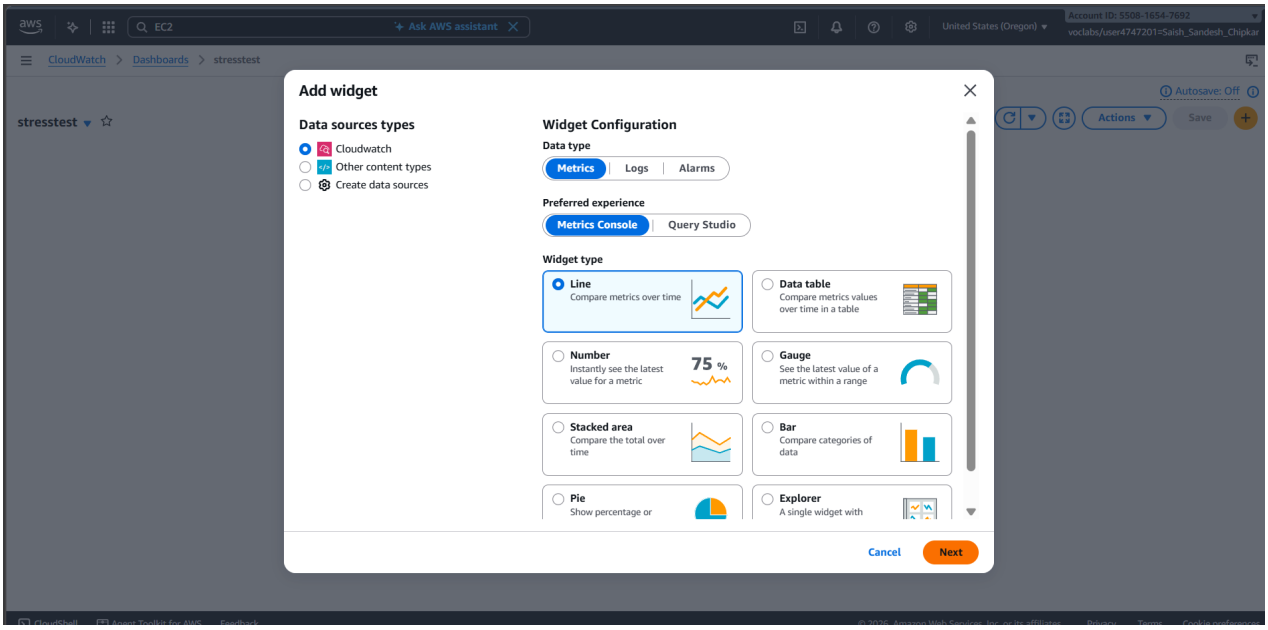


Figure 29: Selecting a CloudWatch metrics line widget

7.10 Step 30: Select the CPUUtilization Metric

In the Add metric graph window, browse the available EC2 metrics.

Locate the EC2 instance named `Stress Test` and select its `CPUUtilization` metric.

The selected metric is displayed in the graph, showing the CPU utilization values recorded over time.

Click Create widget to add the graph to the CloudWatch dashboard.

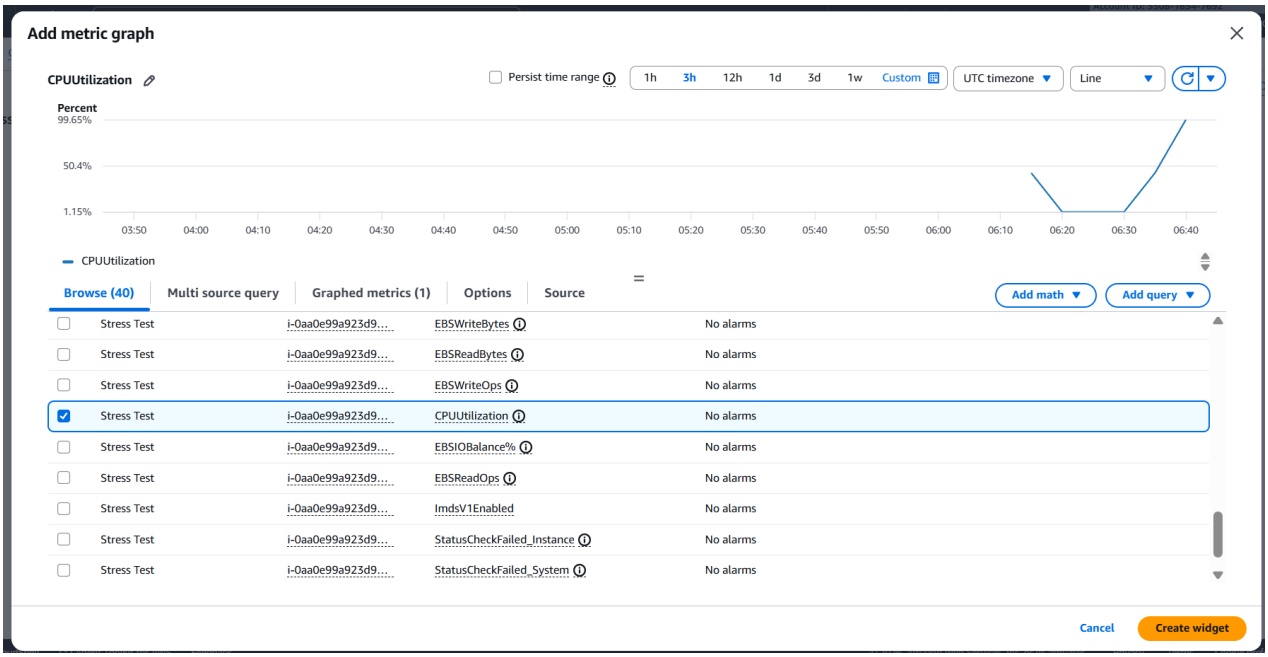


Figure 30: Selecting the EC2 CPUUtilization metric for the dashboard

7.11 Step 31: View the CloudWatch Dashboard

After creating the widget, the CPUUtilization graph appears on the `stresstest` CloudWatch dashboard.

The line graph provides a visual representation of the EC2 instance's CPU utilization over time.

The dashboard can be used to monitor resource utilization and observe changes in CPU performance.

Click the Save button to persist the dashboard configuration.

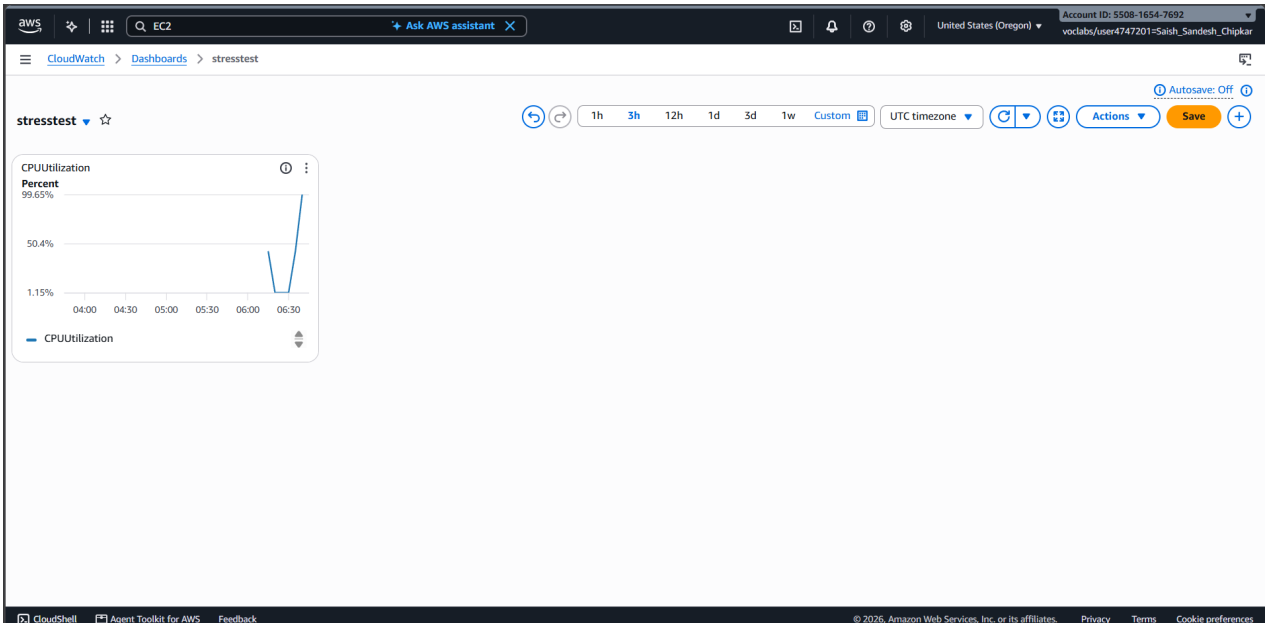


Figure 31: CloudWatch dashboard displaying the CPUUtilization metric

7.12 Final Observation

The experiment successfully demonstrates the integration of Amazon CloudWatch, Amazon SNS and Amazon EC2.

A CPU utilization threshold of 70% was configured using a CloudWatch alarm. An artificial CPU workload was generated using the Stress utility, causing CPU utilization to exceed the configured threshold.

CloudWatch detected the threshold breach, changed the alarm state to ALARM, and triggered the configured actions:

- An email notification was delivered through Amazon SNS.
- The associated EC2 instance was automatically stopped.
- A CloudWatch dashboard widget was created to visualize CPU utilization.

Thus, the experiment demonstrates automated monitoring, notification and resource management using AWS CloudWatch.

8. Observations

- The SNS Standard topic `MyCwAlarm` was created successfully.
- The email subscription was initially pending and was subsequently confirmed.
- The EC2 instance monitored in CloudWatch is named `Stress Test`.
- The selected metric is `CPUUtilization` under the `AWS/EC2` namespace.

- The metric graph shows CPU utilization data, including a displayed value around 42.75% at one point in the selected time range.
- The alarm condition is configured as average CPU utilization greater than 70%.
- The SNS notification and EC2 stop action are selected for the *In alarm* state.

9. Result

The EC2 CPU utilization metric was located and selected in Amazon CloudWatch. An alarm configuration named `LabCPUUtilizationAlarm` was prepared with a static threshold of greater than 70% average CPU utilization. An SNS Standard topic named `MyCwAlarm` was created, and its email subscription was confirmed. The alarm workflow was configured with an SNS email notification and an EC2 stop action for the *In alarm* state. The available screenshots document the process through the final preview stage.

10. Precautions

1. Confirm the SNS email subscription before depending on it for alarm notifications.
2. Verify the EC2 instance ID and metric before creating the alarm.
3. Understand the evaluation period and threshold; a single displayed data point does not necessarily mean the alarm has entered the ALARM state.
4. Use the EC2 stop action carefully because it can interrupt workloads running on the instance.
5. CloudWatch monitors CPU utilization; it does not itself generate CPU load. A separate workload or stress test is required to deliberately increase utilization.